

Machine Organization and ALP



Chapter 2 – Basics of Assembly Language Programming

Alemu W.

awcourseapp@gmail.com

20 March 2015 EC.

Contents



- ✧ Introduction to ALP
- ✧ Components of Assembly Language
- ✧ Assembly Language Directives
- ✧ 80x86 Opcodes
- ✧ Arrays



Introduction

- ✧ Each personal computer has a microprocessor that manages the computer's arithmetical, logical, and control activities.
- ✧ A processor understands only machine language instructions, which are strings of 1's and 0's.
- ✧ Assembly language is a low-level programming language for a computer or other programmable device
 - It is specific to a particular computer architecture in contrast to most high-level programming languages, which are generally portable across multiple systems.
- ✧ Assembly language is converted into executable machine code by a utility program referred to as an assembler like Microsoft Assembler (MASM), Borland Turbo Assembler (TASM), The GNU assembler (GAS), NASM, etc.

Introduction



- ✧ Each family of processors has its own set of instructions for handling various operations such as:
 - Getting input from keyboard
 - Displaying information on screen
 - Performing various other jobs
 - *These set of instructions are called 'machine language instructions'.*
- ✧ However, machine language is too obscure and complex for using in software development.
- ✧ So, the low-level assembly language is designed for a specific family of processors that represents various instructions in symbolic code and a more understandable form.



Advantages of Assembly Language

✧ Understanding of assembly language makes one aware of:

- How programs interface with OS, processor, and BIOS
- How data is represented in memory and other external devices
- How the processor accesses and executes instruction
- How instructions access and process data
- How a program accesses external devices

✧ Other advantages of using assembly language:

- It requires less memory and execution time
- It allows hardware-specific complex jobs in an easier way
- It is suitable for time-critical jobs
- It is most suitable for writing interrupt service routines and other memory resident programs

Basic Features of PC Hardware and Operation



- ✧ The main internal hardware of a PC consists of processor, memory, and registers.
 - Registers are processor components that hold data and address
 - To execute a program, the system copies data and address from the external device into the internal memory
 - The processor executes the program instructions
- ✧ The process through which the processor controls the execution of instructions is referred as the fetch-decode-execute cycle or the execution cycle.
 - Fetching the instruction from memory
 - Decoding or identifying the instruction
 - Executing the instructions

Local Environment Setup



- ✧ Assembly language is dependent upon the instruction set and the architecture of the processor:
 - We will focus on Intel 8086 microprocessor [Recap 8086 microprocessor]
 - Emu8086:
 - Compiles the source code and executes it on emulator step by step
 - Combines an advanced source editor, assembler, disassembler, software emulator (Virtual PC) with debugger, and step by step tutorials.
 - ALU shows the internal work of the central processor unit
 - Emu8086 has a much easier syntax than any of the major assemblers
 - Starting emu8086:
 - Start Emu8086 by clicking it on your computer
 - Select "Samples" from "File" menu
 - Click [Compile and Emulate] button (or press F5 hot key)
 - Click [Single Step] button (or press F8 hot key), and watch how the code is being executed

Assembly Language Basic Syntax



✧ An assembly program can be divided into three sections:

- The data section,
- The bss section, and
- The text or code section

✧ The data Section:

- The data section is used for declaring initialized data or constants
- This data does not change at runtime
- Declare constant values, file names, or buffer size, etc., in this section
- The syntax for declaring data section is – ***section .data***

✧ The bss Section:

- The bss section is used for declaring variables
- The syntax for declaring bss section is – ***section .bss***



Assembly Language Basic Syntax

✧ The text section

- The text section is used for keeping the actual code.
- It must begin with the declaration `global _start`, which tells the kernel where the program execution begins.

✧ The syntax for declaring text section is:

section .text

global _start

_start:

✧ Comments:

- Assembly language comment begins with a semicolon (;)
- It may contain any printable character including blank.
- It can appear on a line by itself, like:
 - ; This program displays a message on screen
 - Or, on the same line along with an instruction, like: `add eax, ebx` ; adds ebx to eax



Assembly Language Statements

- ✧ Assembly language programs consist of three types of statements:
 - Executable instructions or instructions:
 - They are instructions that tell the processor what to do
 - Each instruction consists of an operation code (opcode)
 - Each executable instruction generates one machine language instruction
 - Assembler directives or pseudo-ops:
 - Tell the assembler about the various aspects of the assembly process
 - These are non-executable & do not generate machine language instructions
 - Macros:
 - They are basically a text substitution mechanism



Syntax of Assembly Language Statements

- ✧ Assembly language statements are entered one statement per line
- ✧ Each statement follows the following format:
 - [label] mnemonic [operands] [;comment]
 - The fields in the square brackets are optional.
- ✧ A basic instruction has two parts:
 - The first one is the name of the instruction (or the mnemonic), which is to be executed
 - The second one is the operands or the parameters of the command.
- ✧ Examples:
 - INC COUNT ; Increment the memory variable COUNT
 - MOV TOTAL, 48 ; Transfer the value 48 in the memory variable TOTAL
 - ADD AH, BH ; Add the content of the BH register into the AH register
 - AND MASK1, 128 ; Perform AND operation on the variable MASK1 and 128
 - MOV AL, 10 ; Transfer the value 10 to the AL register



Assembly Language Directives

✧ Assembler directives:

- Instructions used by the assembler at the time of assembling a source program
- The commands/instructions that control the operation of the assembler
- Instructions provided to the assembler, not the processor as the processor has nothing to do with these instructions
- Also known as pseudo-instructions or pseudo-opcode

✧ Task of assembler directives:

- Show the beginning and end of a program provided to the assembler,
- Used to provide storage locations to data,
- Used to give values to variables,
- Define the start and end of different segments, procedures or macros etc. of a program

Assembly Language Directives



✧ Assembler Directives of 8086 microprocessor

- **ASSUME:** Shows the segment name to the assembler
 - It provides information to the assembler regarding the name of the program or data segment for that particular segment.
 - Assume CS: `_DONE` ; This directive specifies that the instruction of the source program is stored in logical segment `_DONE`.
- **DD: Define Double word**
 - This directive allows the initialization of single or multiple data in the form of double words (i.e., 4 bytes).
 - The is used to inform the assembler that the stored data in memory is a double word.
- **DQ: Define Quad words**
 - It is used to initialize quad words (8-bytes) either one or more than one. Thereby informing the assembler that the data stored in memory is quad-word.



Assembly Language Directives

✧ Assembler Directives of 8086 microprocessor

- DT: Define ten bytes
 - It is used to allocate and initialize 10 bytes of a variable.
- DUP: Duplicate
 - Allows initialization of multiple locations and assigning of values to them.
 - This allows storing of repeated characters or variables in different locations.
 - So this permits the storing of these data in memory and creating 8 identical sets in the memory identified as Book.
- DWORD: Double word
 - This directive is used to indicate that the operand is of double word size.
- PROC: Procedure
 - It defines the starting of a procedure/subroutine.
- FAR: This directive is a type specifier that is used by the assembler to declare intersegment call (i.e., call from different segment).

Assembly Language Directives



✧ Assembler Directives of 8086 microprocessor

- NEAR: This is used for intrasegment call i.e., a call within the same segment.
- ENDP: End of procedure
 - This directive shows the termination of a procedure.
- SEGMENT: Beginning of a memory segment.
 - It is used to show the beginning of a memory segment with a specific name.
- ENDS: End of segment
 - This directive defines the termination of a particular memory segment as it is specified by its name.
 - The statements within the segment are nothing but the program code.

Assembly Language Directives



✧ Assembler Directives of 8086 microprocessor

- **EVEN:** It is used to inform the assembler to align the data beginning from an even address.
 - As data specified with an odd starting address requires 2 byte accessing. Thus using this directive, data can be aligned with an even starting address.
- **PTR: Pointer**
 - This directive shows information regarding the size of the operand.
 - This shows legal near jump to BX.
- **PUBLIC:** This directive is used to provide a declaration to variables that are common for different program modules.
- **STACK:** This directive shows the presence of a stack segment.
- **SHORT:** This is used in reference to jump instruction in order to assign a displacement of one byte.
- **THIS:** It is used along with EQU directive for setting the label to either, byte, word or double-word.

Instruction set of 8086 microprocessor



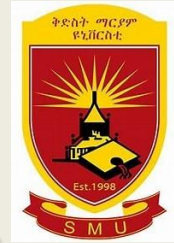
- ✧ The instruction set in 8086 microprocessor are classified as:
- ***Data Transfer Instructions:*** Instructions used for moving the data or address from one place to another – source to destination.
 - Arithmetic Instructions:
 - Logical Instructions
 - String Manipulation Instructions
 - Control Statement Instructions
 - Processor Control Instructions

Instruction Set of 8086 Microprocessor



✧ *Data Transfer Instructions:*

- Data or address can be transferred to either register, memory or I/O ports.
- The instruction groups are:
 - MOV, PUSH, POP, XCHG, XLAT transfer bytes, or words
 - IN, OUT transfer input and output ports
 - LEA, LDS, LES transfer addresses
 - LAHF, SAHF, PUSHF, POPF transfer flag registers
- MOV Instruction:
 - Copies a byte or a word from source to destination
 - Both operands should be of same type either byte or a word.
 - The syntax of this instruction is: ***MOV Destination, Source***
 - Both source and destination operands cannot be memory address
 - The MOV instruction does not affect any value in the flag register.



Instruction Set of 8086 Microprocessor

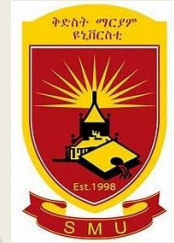
✧ Data Transfer Instructions: MOV Instructions

- The 8086 MOV instruction supports the following operands:

Source	Destination	Example
<i>Reg</i>	<i>Reg</i>	<i>MOV AX, BL</i>
<i>Memory Address</i>	<i>Reg</i>	<i>MOV CX, [2341]</i>
<i>Reg</i>	<i>Memory Address</i>	<i>MOV [7012], AH</i>
<i>Immediate</i>	<i>Reg</i>	<i>MOV AL, 21C</i>
<i>Immediate</i>	<i>Memory Address</i>	<i>MOV [2310], 37B2</i>
<i>Segment Register</i>	<i>Reg</i>	<i>MOV AX, DS</i>
<i>Reg</i>	<i>Segment Register</i>	<i>MOV CS, CX</i>

✧ Example:

```
ORG 100h
MOV AX, a           ;Set AX to 'a' variable
MOV CX, 712h        ;Set CX to 712h
MOV DS, CX          ;Copies value of CX into DS
MOV SI, 25          ;Set SI to 0019 (decimal value of 25)
MOV [105], 6Ah      ;Sets memory address 02712:0069 to 6Ah
MOV 25[BP][SI], AX  ;PA=SS(Shifted left)+a+BP+SI
                   ;Copy data from AX to physical address(PA)
MOV DL, [BX]+6      ;Load data from PA=DS(Shifted left)+BX+6 to DL
RET                ;stops the program
a DW 5C21h
```



Instruction Set of 8086 Microprocessor

✧ Data Transfer Instructions: PUSH Instructions

- The PUSH instruction pushes the data in the stack
- The format of PUSH instruction is: ***PUSH Source***
- It decrements the stack pointer by two and then stores the data from the source operand at the position of the stack pointer
- The source operand can be a general-purpose register, segment register or a memory address but it should be a word.

```
ORG 100h
MOV AX, 1C2Bh    ;Set AX to 1C2B
PUSH AX          ;Set SP=SP-2
                 ;Push AX data to stack location DS:SP

MOV BX, 302Ah    ;Set BX to 302A
PUSH BX          ;SP=SP-2
                 ;Push BX data to stack location DS:SP

RET              ;stops the program
```

Instruction Set of 8086 Microprocessor



❖ Data Transfer Instructions: POP Instructions

- Loads word from the stack pointed by SP and then increments SP by 2
- The format for this instruction is: ***POP destination***
- The destination operand can be a general-purpose register, segment register, or memory address
- The POP instruction does not support CS as a destination operation

```
ORG 100h
MOV AX, 1C2Bh      ;Set AX to 1C2B
PUSH AX            ;Set SP=SP-2 ;Push AX data to stack location DS:SP
MOV BX, 302Ah      ;Set BX to 302A
PUSH BX            ;SP=SP-2          ;Push BX data to stack location DS:SP
POP CX             ;Load the last value stored at stack into CX which is BX
                   ;increment SPby 2 i.e., SP=SP+2
POP [12H]          ;Load the value from location of SP to memory address
DS:0012H
                   ;increment SP by 2
RET                ;stops the program
```

Instruction Set of 8086 Microprocessor



✧ Data Transfer Instructions: Operand Exchange (XCHG)

- XCHG instruction exchanges the contents of the source and destination
- The syntax of instructions is: ***XCHG Destination, Source***
- Both operands should be of the same type either word or a byte
- It does not support segment registers
- The direct exchange of data between memory locations is illegal
- Both operands should be a general-purpose register

```
ORG 100h
MOV AX, a           ;Set AX to 'a' variable
MOV CX, b           ;Set CX to 'b' variable
XCHG AX, CX         ;Exchange words of AX and CX
XCHG AH, CL         ;Exchange bytes of AH and CL
RET                 ;stops the program
a DW 5C21h
b DW 3D05h
```

Instruction Set of 8086 Microprocessor



✧ Data Transfer Instructions: XLAT Instruction

- Used in lookup tables and takes no operands
- Takes the byte number from AL and load the contents of address DS: BX+AL into AL register
- The syntax for this instruction is: ***XLAT*** or ***XLAT Table_Name***
- The BX register contains the offset address of the lookup table and the AL register has a byte number

```
.DATA
TABLE DB '0123456789ABCDEF'
a      DB      13
.code
MOV AL,a           ;AL stores the byte number
MOV BX,OFFSET TABLE ;Sets BX to offset of starting address of the table
XLAT              ;Sets AL to the hexa decimal value of var a
MOV DL,AL
MOV AH,02H        ;Displays the character stored in DL
INT 21H
MOV AH,4CH
INT 21H
RET               ;stops the program
```

Instruction Set of 8086 Microprocessor



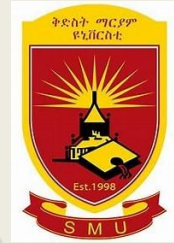
✧ **Data Transfer Instructions: I/O Port Instruction**

✧ ***IN Instruction:***

- Takes the input from the port and transfers that data into the register
- Data transfer takes place between register and I/O device
- The syntax of IN instruction is: ***IN Reg, Port Address***
- The range of port addresses is from 000H to FFFFH
- Example: IN AX, 29H

✧ **OUT Instruction:**

- Outputs the data of register onto a port specified in the instruction
- The syntax of this instruction is: ***OUT Port Address, Register***
- Example OUT 26H, AL



Instruction Set of 8086 Microprocessor

✧ **Data Transfer Instructions:** Address Transfer Instructions

✧ ***LEA Instruction:*** the LEA stands for load Effective address

- It takes the data from the source and copies it to the destination operand
- The destination is always a register whereas the source can be an offset address of a variable or a memory location.
- LEA copies only offset or a memory address to destination register
- The syntax of LEA instruction is: ***LEA Reg, Source***

```
ORG 100h
.MODEL SMALL
.DATA
VAR DB 23h
.code
MOV AX,@DATA ;Sets AX equal to the starting address of data segment
MOV DS,AX    ;Copy AX to DS
LEA DX,VAR   ;Loads the address of VAR
RET          ;stops the program
```



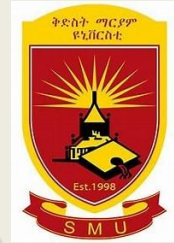
Instruction Set of 8086 Microprocessor

✧ Data Transfer Instructions: Address Transfer Instructions

✧ LDS Instruction:

- Stores four consecutive memory locations into a specified destination register and a DS register.
- The format of LDS instruction is: ***LDS Reg, Memory Address***
- The word from first two memory locations is loaded into a register and the word from the next two memory locations gets stored to DS register

```
ORG 100h
.MODEL SMALL
.DATA
VAR DB 23h
.code
LDS AX, VAR
RET
```



Instruction Set of 8086 Microprocessor

✧ Data Transfer Instructions: Address Transfer Instructions

✧ LES Instruction:

- It loads data from first two memory locations to a specified register
- The data of the next two memory location goes to ES register
- The syntax of LES instruction is: ***LES Reg, Memory Address***

```
ORG 100h
.MODEL SMALL
.DATA
NUM DB 5Ch
.code
LES SI, NUM
MOV AX, 0710H
MOV DS, AX
LES BX, [8H]
RET
```

✧ Read about ***Other 8086 Data Movement Instructions*** and exercise it in your lab.

Instruction Set of 8086 Microprocessor



✧ Logical Instructions – AND Logical Instruction

- Perform logical AND operation between two operands
- The source can be an immediate, register, or a memory location and the destination can be either a register or a memory location
- It ANDs each bit of source operand with the destination operand and stores the result back into the destination operand
- The syntax of AND instruction: AND Destination, Source

```
ORG 100h
.MODEL SMALL
.CODE
MOV BX, 3527
MOV CX, 2968
AND BX, CX
RET
```

Instruction Set of 8086 Microprocessor



✧ Logical Instructions – OR Logical Instruction

- Performs the OR operation between two operands and stores the result back into the destination operand
- The destination operand can be a register or a memory location whereas the source can be immediate, register, or a memory location
- The OR instruction clears the CF and OF flags to 0 and update PF, ZF, and SF flags
- The syntax of OR instruction: OR Destination, Source

```
ORG 100h
.MODEL SMALL
.DATA
VAR_1 DW 3527
.CODE
MOV AX, 0700h
MOV DS, AX
MOV CX, 2968
OR CX, [102h]
RET
```

Instruction Set of 8086 Microprocessor



✧ Logical Instructions: – XOR Logical Instruction

- Performs the XOR operation between bits of source and destination operands
- The source operand can be either a register or memory address whereas destination operand can be immediate, register or memory location
- The syntax of XOR instruction: XOR Destination, Source

```
ORG 100h  
.MODEL SMALL  
.CODE  
MOV AH, 68H  
XOR AH, 9CH  
RET
```

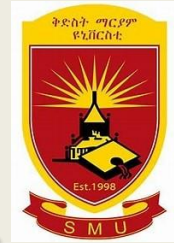
Instruction Set of 8086 Microprocessor



✧ Logical Instructions: – NEG 2's Complement Instruction

- Compute the 2's complement of the destination operand
- The destination can be a register or memory address
- It can change AF, PF, ZF and OF flags of flag register
- The syntax of XOR instruction: NEG Destination

```
ORG 100h
.MODEL SMALL
.CODE
MOV AX, 25H    ;Sets AX to 25H
NEG AX         ;Compute 2's complement of data in AX
MOV DX,0711H
MOV DS,DX      ;Sets Data Stack pointer to 0711H
NEG [9H]       ;Compute 2's complement of value stored at memory address DS:0009H
RET            ;Stops the program
```



Instruction Set of 8086 Microprocessor

✧ Logical Instructions: – CMP Compare Instruction

- Compares the data of the two operands and depending upon the result sets the flag
- Compares the two operands by computing the difference of two operands and sets CF, ZF and SF flags
- The syntax of CMP instruction: CMP Destination, Source
- The source and destination can be a register, immediate number, or a memory address
- Both operands cannot be a memory address in the same instruction

```
ORG 100h
.MODEL SMALL
.CODE
MOV AX, 0025H ;Sets AX to 25H
MOV BX, 00A5H ;Sets BX to A5H
CMP AX, BX    ;Compare the contents of AX to BX
RET           ;Stops the program
```

	CF	ZF	SF	
Dest=Source		0	1	0
Dest>Source		0	0	0
Dest<Source		1	0	1

Instruction Set of 8086 Microprocessor



✧ **Arithmetic Instructions:** Addition Instructions

- **ADD:** adds the data of destination and source operand and stores the result in destination.
 - Both operands should be of same type i.e. words or bytes
 - Can affect AF, CF, OF, PF, SF, ZF flags depending upon the result
 - It supports following operands:

Source	Destination	Example
Register	Register	ADD AX, BX
Register	Immediate	ADD CL, 2DH
Register	Memory Address	ADD [204CH], AL ADD [BP][SI]+23, DX
Memory Address	Register	ADD CL, [BX+15]

Instruction Set of 8086 Microprocessor



✧ Arithmetic Instructions: Addition Instructions

```
ORG 100h
.MODEL SMALL
.CODE
MOV AL, 10H      ;Sets AL to 10H
MOV BH, 75H      ;Sets BX to 23H
ADD AL, BH        ;Store the sum of AL and BX in AL
MOV CX, 0F21Ah   ;Set CX to 0F21Ah
ADD [0154H], CX  ;Store sum of CX data and data at memory
address DS:0154 into the same memory address
MOV AH, 9FH      ;Sets AH to 9FH
ADD AH, BH        ;Store sum of AH and BH into BH
RET              ;stops the program
```



Instruction Set of 8086 Microprocessor

✧ **Arithmetic Instructions:** Addition Instructions

- **ADC:** adds the carry flag bit to the sum of two operands

```
ORG 100h
.MODEL SMALL
.CODE
MOV AL, 7DH      ;Sets AL to 7DH
MOV BH, 0F5H     ;Sets BH to F5H
ADC AL, BH       ;Store the sum of AL and BH in AL
MOV CX, 0F21Ah   ;Set CX to 0F21Ah
MOV [0154H], CX  ;CX=CX+ DS:0154H
ADC AL, [0154H]  ;AL=AL+ DS:0154H
RET              ;stops the program
```



Instruction Set of 8086 Microprocessor

✧ Arithmetic Instructions: Addition Instructions

■ INC Instruction:

- It is an increment instruction which takes only one operand
- The INC instruction adds 1 to the contents of destination operand
- It can affect AF, OF, PF, SF and ZF flags

```
ORG 100h
.MODEL SMALL
.CODE
MOV AL, 7DH    ;Sets AL to 7DH
INC AL         ;AL=AL+1
RET            ;stops the program
```

- *Read about AAA and DAA Addition instructions in 8086 microprocessors.*



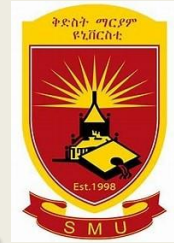
Instruction Set of 8086 Microprocessor

✧ Arithmetic Instructions: Subtraction Instructions

- SUB: takes two operands and then subtract the data in the source operand from the data of destination operand and then store the result back to the destination operand.
 - Both operands should be either in bytes or words
 - Supported operands:

<i>Source</i>	<i>Destination</i>	<i>Example</i>
<i>Register</i>	<i>Register</i>	<i>SUB CH, AL</i>
<i>Register</i>	<i>Immediate</i>	<i>SUB AX, 16H</i>
<i>Register</i>	<i>Memory Address</i>	<i>SUB [SI], AX SUB 19[BP][DI], CX</i>
<i>Memory Address</i>	<i>Register</i>	<i>SUB AL, 10[CX]</i>

```
ORG 100h
MOV AX, 2506    ;Sets AX to 2506
MOV BX, 1647    ;Sets BX to 1647
SUB AX, BX      ;AX=AX-BX
SUB [SI], AX    ;DS:SI=DS:SI-AX
RET            ;Stop the program
```



Instruction Set of 8086 Microprocessor

✧ Arithmetic Instructions: Subtraction Instructions

✧ SBB:

- Subtract the source data from destination's data and subtracts the carry flag bit from their result and then store the result in Destination operand
- The flags AF, CF, OF, PF, SF and ZF can be affected by this instruction

```
ORG 100h
MOV AX, 2506      ;Sets AX to 2506
MOV BX, 1647      ;Sets BX to 1647
SBB AX, BX        ;AX=AX-BX
MOV SI, 0700H     ;Set SI to 0700H
SBB [SI], AX       ;DS:SI=DS:SI-AX
RET               ;Stop the program
```

✧ DEC: subtracts 1 from the destination operand and loads the result back into the same destination.

✧ *Read about AAS and DAS subtraction instructions*



Instruction Set of 8086 Microprocessor

✧ Arithmetic Instructions: Multiplication Instructions

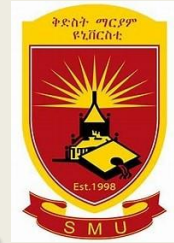
✧ MUL: multiplication of two unsigned numbers

- Byte with Byte multiplication: one operand resides in an AL register and the other is source. Source can be a register or memory address.

```
ORG 100h
.CODE
MOV AL, 35
MOV BH, 15
MUL BH
RET
```

```
ORG 100h
.MODEL SMALL
.DATA
VAR_1 DW 12DAH
VAR_2 DW 3F24H
.CODE
MOV AX, VAR_1
MOV BX, VAR_2
MUL BX
RET
```

- Word with Word multiplication: one operand is loaded in AX register and the source should be a 16-bit register or a memory address.
 - The two words of 16-bits on multiplication can produce a 32-bit word, then the lower bytes of word are stored in AX and higher bytes in DX register.³⁹



Instruction Set of 8086 Microprocessor

✧ Arithmetic Instructions: Multiplication Instructions

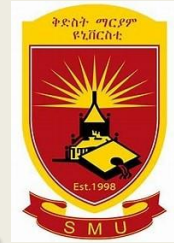
✧ IMUL: allows the multiplication of two signed operands

- When the operand is a byte, it is multiplied with AL register and when it is a word, it is multiplied with AX register
- If the product of multiplier and multiplicand produce result that fits into the destination register DX and AX with some of the bits left unused. Then these unused bits are filled with the copies of signed bit and clear CF and OF flags to zero.

```
ORG 100h
.MODEL SMALL
.CODE
MOV AL, 2AH      ;Load 1st operand to AX
MOV BX, -26CH    ;Load 2nd operand in BX
IMUL BX
RET
```

- ***Read about AAM instruction.***

Instruction Set of 8086 Microprocessor



✧ Arithmetic Instructions: Division Instructions

✧ DIV: Performs the division of two unsigned operands

- The denominator resides in a source operand and it should not be immediate, but can be register or a memory location
- The division can be:
 - Byte with Byte: the nominator and denominator operands are bytes.
 - The nominator resides in the AL register and AH is set to zero
 - Quotient is stored in AL and the remainder in AH register
 - Word with Word: the AX register holds the numerator.
 - Quotient is stored in AX register & remainder goes to the DX register
 - Word with Byte: The numerator is a 16-bit word stored in AX
 - AL contains the quotient and AH will contain the remainder
 - Doubleword with word: AX and DX stores the numerator.
 - The most significant bits resides in the DX register and the least significant bits of numerator are in the AX register.

Instruction Set of 8086 Microprocessor



✧ Arithmetic Instructions: Division Instructions

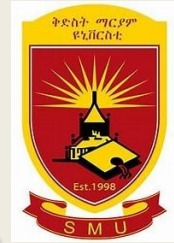
✧ DIV:

```
ORG 100h
.MODEL SMALL
.DATA
NUM_1 DB 0F2H
NUM_2 DB 4H
.CODE
MOV BH, NUM_2 ;Load numerator in BH
MOV AL, NUM_1 ;Load denominator in AL
DIV BH ;Divide BH by AL
RET
```

```
ORG 100h
.MODEL SMALL
.DATA
NUM_1 DW 0F213H
NUM_2 DW 41A8H
.CODE
MOV AX, NUM_1 ;Load numerator in AX
DIV NUM_2 ;Divide AX by NUM_2
RET
```

```
ORG 100h
.MODEL SMALL
.DATA
NUM_1 DW 1B25H
NUM_2 DB 24H
.CODE
MOV AX, NUM_1 ;Load denominator in AX
DIV NUM_2 ;Divide AX by NUM_2
RET
```

```
ORG 100h
.MODEL SMALL
.DATA
NUM_1 DW 2413H
.CODE
MOV AX, 5670 ;Load lower bytes of
              numerator in AX
MOV DX, 4531 ;Load high bytes of
              numerator in DX
DIV NUM_1 ;Divide AX by NUM_1
RET
```



Instruction Set of 8086 Microprocessor

✧ Arithmetic Instructions: Division Instructions

✧ DIV: Divide Error or Overflow Error

- The emulator would generate a type 0 interrupt if:
 - In case of a byte with byte and word with byte division, the quotient is greater than FFH
 - In case of a word with word and doubleword by word division, if the quotient is greater than FFFFH.
 - When the denominator is 0

```
ORG 100h
.MODEL SMALL
.DATA
NUM_1 DW 5BBDH
NUM_2 DB 21H
.CODE
MOV AX, NUM_1 ;Load denominator in AX
DIV NUM_2 ;Divide AX by NUM_2
RET
```

Instruction Set of 8086 Microprocessor



✧ Arithmetic Instructions: Division Instructions

✧ IDIV: a division operation between two signed numbers

- The source operand in the instruction is a signed divisor and can be specified using any addressing mode except immediate mode of addressing
- **Byte with Byte:** the numerator is stored in AL and its sign extended to AH register
 - The quotient goes to AL register and remainder is in AH register
 - The quotient value should be in between +127 to 127 range
- **Word with Word:** the numerator is in AX register and the leftover most significant bits are filled with copies of signed bit
 - The division generates quotient in AX and remainder in DX
 - The quotient range is -32767 to +32767
- **Word with Byte:** the AX register will store the numerator
- **Doubleword with word:** DX register will store the most significant bits of the numerator and least significant bits are in AX register.
- AX will store quotient and DX will store remainder



Instruction Set of 8086 Microprocessor

✧ Arithmetic Instructions: Division Instructions

✧ IDIV: a division operation between two signed numbers

```
ORG 100h
.MODEL SMALL
.DATA
NUM_1 DD 0C250A91H
NUM_2 DW -0B25H
NUM_3 DB 24H
.CODE
;Byte by Byte Signed Division
MOV AL, NUM_3
MOV BL,3
IDIV BL

;Word by word Signed Division
MOV AX, -265H
IDIV NUM_2
```

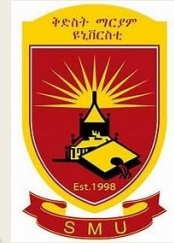
```
;Doubleword by single word signed
division
MOV AX,NUM_1
MOV DX, NUM_1+2
MOV BX, 0A234H
IDIV BX
RET
```

Instruction Set of 8086 Microprocessor



✧ Conditional Branch Instructions:

- There are two types of branches/jumps: conditional and unconditional
- Unconditional Branch Instructions: the program counter jumps to the label address provided within the instruction
- Conditional Branch Instructions: execution is based on some condition
 - Checks one or more flag conditions and transfers the control to a new memory location.
 - There are two types of jumps: Far and Near
 - In the far jumps, the program counter jumps to the memory location which lies outside the current code segment
 - In near jumps, the IP points to the memory address inside the current code segment and that is why the CS register remains unchanged in near jumps.
 - The syntax of these instructions is: **Opcode LABEL**



Instruction Set of 8086 Microprocessor

✧ Examples for 8086 Conditional Branch Instructions

Instructions	Operation	Testing Condition
JA/JNBE	Jump if Above/Not Below or Equal	C=0 and Z=0
JAE/JNB/JNC	Jump if Above or Equal/ Jump if Not Below/Jump if No Carry	C=0
JB/JNAE	Jump if Below/ Jump if Not Above or Equal	C=1
JBE/JNA	Jump if Below or Equal/ Jump if Not Above	(C or Z) =1
JC	Jump if Carry	C=1
JNC	Jump if Not Carry	C=0
JCXZ	Jump if the CX register=0	CX=0
JE/JZ	Jump if Equal/Jump if Zero	Z=1
JG/JNLE	Jump if Greater/Jump if Not Less Than or Equal	$((S \text{ xor } O) \text{ or } Z) = 0$
JGE/JNL	Jump if Greater or Equal/Jump if Not Less Than	$(S \text{ xor } O)=0$
JL/JNGE	Jump if Less Than/Jump if Not Greater Than or Equal	$(S \text{ xor } O)=1$
JLE/JNG	Jump if Less than or Equal/Jump if Not Greater	$((S \text{ xor } O) \text{ or } Z) = 1$
JNE/JNZ	Jump if Not Equal/ Jump if Not Zero	Z=0
JNP/JPO	Jump if Not Parity/Jump if Parity Odd	P=0



Instruction Set of 8086 Microprocessor

✧ Example:

```
ORG 100h
.MODEL SMALL
.DATA
NUM_1 DB 23H
NUM_2 DB 93H
MSG DB "Equal Numbers$"
MSG1 DB "Number 1 is less than Number 2$"
MSG2 DB "Number 1 is greater than Number 2$"

.CODE
MOV AX, @DATA
MOV DS, AX

MOV AH, NUM_1
MOV CH, NUM_2
CMP AH, CH
JE L1 ;If AH and CH are equal
JB L2 ;If AH is less than CH
JA L3 ;If AH is greater than CH
```

```
L1:
MOV DX, OFFSET MSG
MOV AH, 09H
INT 21H
RET

L2:
MOV DX, OFFSET MSG1
MOV AH, 09H
INT 21H
RET

L3:
MOV DX, OFFSET MSG2
MOV AH, 09H
INT 21H
RET
```

Instruction Set of 8086 Microprocessor







